

Deployment Readiness Checklist

Version: 1.0.0 Last updated: 2026-08-18

Not every item applies to every project. Skip anything tagged with an Applicability note that doesn't match your stack. Priority legend: ● Critical · ● Recommended · ● Nice-to-have.

1. Project / Code Readiness

- ☐ ● **No debug/dev mode flags left on in production build** — Verify: any `DEBUG=true`, verbose error pages, or dev-only middleware is disabled. Why: debug mode routinely leaks stack traces, source paths, and internal state to attackers.
- ☐ ● **All TODO/FIXME/temporary hacks reviewed before merge** — Verify: grep the codebase for `TODO`, `FIXME`, `HACK` before tagging a release. Why: these mark known-incomplete logic that's easy to forget once shipped.
- ☐ ● **No dead code or leftover debug statements** — Verify: no commented-out blocks, `console.log / print` debug lines in the diff. Why: clutter and occasional accidental data exposure in logs.
- ☐ ● **Code passes linter with no warnings on main branch** — Verify: CI lint step is green. Why: catches whole classes of bugs before they reach runtime.

2. Dependency Management

- ☐ ● **Vulnerability audit run before every deploy** — Verify: `npm audit / pip-audit` / equivalent shows no unresolved high/critical CVEs. Why: known vulnerabilities in dependencies are one of the most common breach vectors.
- ☐ ● **No abandoned packages with known CVEs** — Verify: check last-publish date and open security advisories for critical dependencies. Why: unmaintained packages won't get patched even after a CVE is public.
- ☐ ● **Dependency scan automated in CI** — Verify: scan runs on every push, not just manually. Why: catches new CVEs disclosed after your last manual check.
- ☐ ● **Lockfile committed, versions pinned** — Verify: `package-lock.json / poetry.lock` / equivalent is committed and used in CI. Why: prevents "works on my machine" drift and supply-chain surprises from unpinned transitive updates.

3. Environment Configuration

- ☐ ● **Staging and production use fully separate credentials/databases** — Verify: no shared DB, API keys, or service accounts between environments. Why: the most common way staging bugs or test data corrupt production.
- ☐ ● **Correct env vars set per environment** — Verify: no dev/test values (e.g. test API keys, localhost URLs) present in the production config. Why: silent wrong-environment behavior is hard to detect after the fact.
- ☐ ● **.env.example kept current** — Verify: every required var has a placeholder entry, no real values. Why: keeps onboarding and redeploys from breaking due to undocumented config.

4. Secrets & Credentials

- ☐ ● **No secrets in code or git history** — Verify: no API keys, DB passwords, or tokens committed anywhere, ever. Why: git history is effectively public/permanent even in "private" repos.
- ☐ ● **.env gitignored; secrets via env vars or a secrets manager** — Verify: `.env` is in `.gitignore` before the first commit. Why: this is the #1 accidental-leak vector.
- ☐ ● **Any previously-committed secret purged from git history** — Verify: use `git filter-repo / BFG`, not just a delete commit, and rotate the exposed credential regardless. Why: deleting a file doesn't remove it from history — it's still retrievable.
- ☐ ● **Secret scanning in CI** — Verify: `gitleaks/truffleHog` (or similar) runs on every push. Why: catches leaks before merge instead of after — proactive vs. the cleanup step above.
- ☐ ● **Private/restricted DB key used, never a public or default one** — Verify: no default credentials from a tutorial or scaffold left in place. Why: default creds are the first thing automated scanners try.

5. Security

- ☐ ● **Every endpoint checks resource ownership, not just login status** — Verify: "is this user logged in" is never treated as sufficient — check they own *this specific* resource. Why: this is the core defense against one user reaching another's data.
- ☐ ● **Never trust IDs from the frontend** — Verify: e.g. `/user/123/invoice/456` — backend re-verifies the logged-in user actually owns invoice 456, regardless of what the URL says. Why: frontend routing is not an access control mechanism.
- ☐ ● **Unguessable IDs (UUIDs) instead of sequential numbers** — Verify: primary identifiers exposed in URLs/APIs aren't incrementable. Why: sequential IDs let attackers enumerate and probe every record.
- ☐ ● **Row-level security enabled at the database level** — Verify: DB itself enforces that queries only return rows belonging to the requesting user. Why: a backup layer that still protects data even if application logic has a bug.
- ☐ ● **Parameterized queries used everywhere** — Verify: no string-concatenated SQL anywhere in the codebase. Why: the actual fix for SQL injection — not an optional style choice.
- ☐ ● **User content escaped/sanitized before rendering** — Verify: any user-generated content shown in the UI is escaped. Why: prevents XSS.
- ☐ ● **CORS configured explicitly, not wide open** — Verify: `Access-Control-Allow-Origin` is not `*` on any endpoint handling authenticated requests. Why: overly permissive CORS is a common, easily-missed way to let malicious sites make authenticated requests on a user's behalf.
- ☐ ● **File uploads restricted and scanned** — Verify: type, size, and content are checked; never trust the file extension alone. Why: unrestricted uploads are a common path to remote code execution or stored malware.
- ☐ ● **Mass-assignment protection** — Verify: only explicitly allowed fields are accepted/updated (a user can't send `{"isAdmin": true}` on a profile update). Why: prevents silent privilege escalation through over-permissive update endpoints.
- ☐ ● **API responses trimmed to what's needed** — Verify: responses don't include password hashes, internal flags, or other fields the frontend doesn't use. Why: minimizes what's exposed if a response is ever intercepted or over-shared.
- ☐ ● **Error messages don't leak internals** — Verify: users never see stack traces or file paths. Why: gives attackers a map of your internals for free.
- ☐ ● **SSRF protection on outbound server requests** — Verify: if the server fetches URLs on a user's behalf, internal/private IP ranges are blocked. Why: prevents an attacker using your server to reach internal infrastructure.

6. Authentication & Authorization

- ☐ **Passwords hashed with bcrypt/argon2** — Verify: never stored in plaintext or with a weak/fast hash (MD5, SHA1). Why: standard, non-negotiable practice.
- ☐ **Cookies set Secure, HttpOnly, SameSite=Strict** — Verify: check response headers on auth cookies. Why: mitigates cookie theft via XSS and cross-site request forgery.
- ☐ **CSRF tokens on all state-changing requests** — Verify: POST/PUT/DELETE endpoints require a valid token. Why: without this, a malicious site can trigger actions using a logged-in user's session.
- ☐ **Sessions/tokens expire and can't be reused after logout** — Verify: logout actually invalidates the token server-side, not just clears it client-side. Why: a stolen token should have a limited useful life.
- ☐ **Admin/internal routes require server-side role checks** — Verify: never gated only by hiding a button on the frontend. Why: frontend-only gating is trivially bypassed.
- ☐ **Rate limiting on login/signup/reset/OTP endpoints** — Verify: repeated requests from the same source are throttled. Why: these are the endpoints most targeted by automated abuse.
- ☐ **Brute-force lockout after repeated failed attempts** — Why: slows down credential-stuffing attacks.
- ☐ **CAPTCHA/bot protection on high-abuse endpoints** — Applicability: public signup/contact forms.
- ☐ **Password visibility toggle on auth forms** — Applicability: Frontend. Why: reduces login errors, minor UX win.

7. Database

- ☐ **Migrations are backward-compatible / have a rollback path** — Verify: a migration can be reverted without data loss, and old code can run against the new schema briefly during rollout. Why: the single most common cause of a botched deploy is a migration that can't be undone.
- ☐ **Database not exposed directly to the public internet** — Verify: DB only reachable from the app's network/VPC, not a public port. Why: removes an entire class of direct-attack risk.
- ☐ **Least-privilege DB user for the app** — Verify: the application's DB account doesn't have admin/superuser rights. Why: limits damage if the app layer is ever compromised.
- ☐ **Indexes reviewed for critical queries** — Why: prevents slow queries from becoming outages under real load.
- ☐ **Connection pool limits set appropriately** — Why: prevents the app from exhausting DB connections under load.

8. API / Backend

- ☐ **All input validated on the backend, always** — Verify: never rely on frontend validation for security, only UX. Why: frontend checks are trivially bypassed by anyone calling the API directly.
- ☐ **Global error handler; no unhandled exceptions crash the process** — Why: one bad request shouldn't take down the whole service.
- ☐ **API versioning strategy in place** — Applicability: APIs expecting future breaking changes.

9. Frontend

- ☐ **Custom 404 page** — Why: better UX and avoids leaking framework default error pages.
- ☐ **Form success/error states clearly shown** — Why: users need feedback that an action worked or didn't.
- ☐ **Loading states for async content** — Why: prevents the appearance of a frozen/broken page.
- ☐ **Confirmation modals for destructive actions** — Applicability: any delete/irreversible action.
- ☐ **General polish bundle** — dark mode toggle, sticky header, mobile menu, back-to-top button, scroll progress bar, copy button, print stylesheet, expandable FAQ, floating contact widget. Applicability: content/marketing-facing sites.

10. Testing

- ☐ **Test suite passes and gates deployment in CI** — Verify: a failing test blocks deploy, not just a warning. Why: the whole point of tests is meaningless if a failure doesn't stop a release.
- ☐ **Critical user paths covered by at least basic tests** — e.g. signup, login, checkout, core CRUD flows.
- ☐ **Load/stress test before high-traffic launches** — Applicability: expected high-traffic or public launches.

11. Error Handling

- ☐ **Global error boundary/handler** — Verify: no unhandled error exposes a raw stack trace to the user. Why: same class of leak as debug mode — internal details shouldn't reach the client.
- ☐ **Custom 500/error page** — Why: better UX than a framework default error screen.

12. Logging

- ☐ **Passwords/tokens/PII never logged in plaintext** — Why: logs are often less protected than the database itself and widely accessible internally.
- ☐ **Auth failures and unusual access patterns logged** — Why: needed to detect and investigate probing/attacks after the fact.
- ☐ **Log retention policy defined** — Why: indefinite retention is a liability; too-short retention loses forensic value.

13. Monitoring / Observability

- ☐ **Uptime monitoring on production** — Why: you should find out about downtime before your users tell you.
- ☐ **Alerting on repeated 401/403 responses** — Why: a signal someone is actively probing for access.
- ☐ **Analytics/usage tracking configured** — Applicability: user-facing products.
- ☐ **UTM tracking on campaign links** — Applicability: marketing-driven launches.

14. Performance

- ☐ ● **Caching strategy for expensive/repeated queries or requests**
- ☐ ● **CDN in front of static assets**
- ☐ ● **N+1 query check on main data-heavy pages**
- ☐ ● **Frontend bundle size checked** — Applicability: Frontend.

15. Accessibility

- ☐ ● **Alt text on all meaningful images**
- ☐ ● **Skip-to-content link** — Why: lets keyboard/screen-reader users bypass repeated nav.
- ☐ ● **Basic keyboard navigability spot-check**

16. SEO

Applicability: content/marketing-facing sites

- ☐ ● **Unique page titles and meta descriptions**
- ☐ ● `robots.txt` **present and correct**
- ☐ ● **Social share image (OG tags), local schema markup**

Content/marketing items (Applicability: marketing/content sites only — not applicable to APIs, backend services, or internal tools): - ☐ ● Breadcrumbs - ☐ ● Internal links - ☐ ● Thank-you page after form submission - ☐ ● Site search - ☐ ● 5 FAQs answered on-page - ☐ ● Case studies - ☐ ● Real customer reviews - ☐ ● Team photo - ☐ ● Stated response-time promise - ☐ ● Maps + directions

17. HTTPS / TLS

- ☐ ● **HTTPS enforced everywhere, HSTS enabled**
- ☐ ● **HTTP → HTTPS redirect forced at server/CDN level**
- ☐ ● **TLS certificate auto-renewal configured** — e.g. Let's Encrypt.

18. Domain & DNS

- ☐ ● **DNS propagation verified before go-live**
- ☐ ● **Domain auto-renewal enabled** — Why: avoids accidental expiry and hijack risk.

19. Hosting / Infrastructure

- ☐ ● **Health check endpoint configured** — Why: needed by load balancers/orchestrators to know if an instance is alive.
- ☐ ● **Resource limits / autoscaling caps set** — Why: prevents runaway cost or an unbounded crash loop.

20. CI/CD

- ☐ ● **Staging environment mirrors production configuration** — Why: a staging pass that doesn't reflect prod config gives false confidence.
- ☐ ● **Deployment requires an explicit approval step** — Why: prevents accidental auto-deploy from the wrong branch.

21. Containers

Applicability: Docker/Kubernetes projects

- ☐ ● **Container runs as a non-root user**
- ☐ ● **Base image scanned for vulnerabilities**
- ☐ ● **Multi-stage build to minimize final image size**

22. Cloud Configuration

Applicability: AWS/GCP/Azure projects

- ☐ ● **IAM roles follow least privilege**
- ☐ ● **Billing/cost alert configured**

23. Backups & Disaster Recovery

- ☐ ● **Backups encrypted**
- ☐ ● **Restore process actually tested** — Why: a backup that's never been restored isn't a verified backup.
- ☐ ● **Backup automation scheduled**, not manual-only.

24. Privacy / Data Handling

- ☐ ● **Privacy Policy published and accurate to what's actually collected**
- ☐ ● **Terms of Service published**
- ☐ ● **Cookie consent banner if using analytics/tracking**
- ☐ ● **Regional compliance addressed** (e.g. DPDP Act 2023 in India, GDPR in EU) — consent mechanisms, data retention limits. *Context-dependent: applies based on your users' jurisdiction, not your own.*

☐ ● **Sensitive data (PII/financial/health) encrypted at rest**

25. Production Configuration

☐ ● **Debug mode confirmed OFF**

☐ ● **Security headers set** — `Content-Security-Policy` , `X-Content-Type-Options: nosniff` , `X-Frame-Options: DENY` , `Referrer-Policy` , `Strict-Transport-Security` .

26. Deployment Process

☐ ● **Zero-downtime deploy strategy**, or an accepted downtime window is communicated in advance.

☐ ● **Migration ordering is backward-compatible during rollout** — old running code shouldn't break against a mid-rollout schema.

27. Smoke Testing After Deployment

☐ ● **Health endpoint hit and confirmed after every deploy**

☐ ● **Manually try accessing another user's data by changing the URL/ID** — Why: the single most important manual access-control test.

☐ ● **Confirm admin routes are unreachable and non-indexable for non-admins**

28. Rollback / Recovery

☐ ● **Documented rollback procedure exists**

☐ ● **Previous working version/image readily available to redeploy**

29. Documentation

☐ ● **README explains setup, env vars, and how to run/deploy**

☐ ● **Runbook/on-call notes** — who owns what if something breaks.

30. Post-Deployment Maintenance

☐ ● **Dependency update cadence defined** — e.g. monthly audit.

☐ ● **Uptime/log review cadence defined**